

Kompakte arc42-Dokumentation

SEKA - Semesterprojekt

Implementierter Analyseprototyp | Stand 16. September 2026

Zweck	Architektur, Entscheidungen, Risiken und Nachweise des implementierten Analyseausschnitts
System	React-Weboberfläche, sechs Spring-Boot-Services und Docker Compose
Abgrenzung	Keine physikalische Simulation und keine vollständige Migration der Unternehmensplattform

Inhalt

1. Einführung und Ziele
 2. Randbedingungen
 3. Kontextabgrenzung
 4. Lösungsstrategie
 5. Bausteinsicht
 6. Laufzeitsicht
 7. Verteilungssicht
 8. Querschnittliche Konzepte
 9. Entwurfsentscheidungen
 10. Qualitätsanforderungen und Nachweise
 11. Risiken und Retrospektive
 12. Glossar und Quellen
- Anhang: vier UML-Sichten

1. Einführung und Ziele

Die Fallstudie beschreibt eine gewachsene Unternehmensplattform mit gemeinsamem Datenmodell, gekoppelten Komponenten und einer blockierenden, schwer beobachtbaren Motoranalyse. Dieser Prototyp realisiert den Analyseausschnitt: Ein Ingenieur speichert eine Konfiguration, startet vier Analysealgorithmen und sieht Fortschritt, Equipmentresultate und Gesamtergebnis. Ein fehlgeschlagener Schritt lässt sich nach Beseitigung der Ursache wiederholen.

Priorisiert werden fachlich verständliche Grenzen, nachvollziehbare Ergebnisse, begrenzte Wartezeiten und eine reproduzierbare Demo. Eine physikalische Simulation und die Migration von ERP, CRM oder Order Fulfillment liegen außerhalb dieses Ausschnitts.

2. Randbedingungen

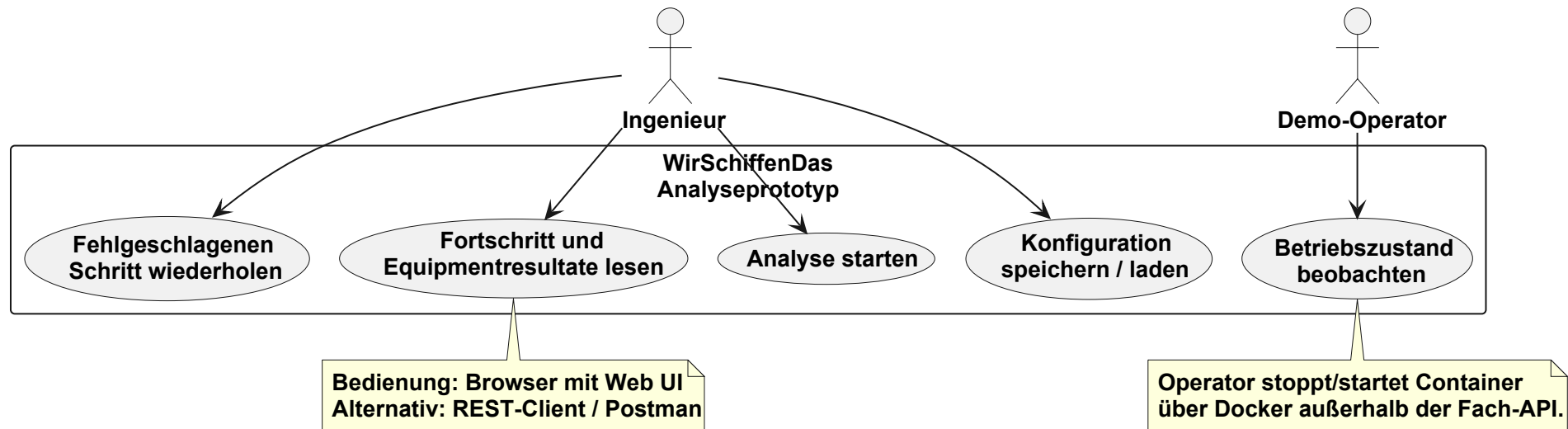
Übung 5 verlangt persistente Konfiguration, mindestens vier Analyse-Microservices, REST, Choreographie, proaktive Rückmeldungen und Circuit-Breaker-Verhalten. Übung 8 verlangt vier UML-Sichten, Entscheidungen, Code-Walkthrough, Demo und Fazit. Docker/Compose erfüllt eine technische Alternative aus Übung 8; Resilience4j ergänzt sie.

Implementiert sind Java 21/Spring Boot, React/MUI, REST/JSON, zwei getrennte H2-Datenbanken, Resilience4j und Docker Compose. Die Algorithmen prüfen fünf Equipmentfelder: leer oder INVALID ist ungültig. Diese Demonstrationsregeln haben keine physikalische Aussagekraft.

3. Kontextabgrenzung

Der Ingenieur bedient die Weboberfläche im Browser: Konfiguration, Analyse, Status, Ergebnisse und Retry. Der Demo-Operator stoppt und startet Services über Docker. Postman ist eine zusätzliche Bedienmöglichkeit. ERP und CRM sind keine laufenden Nachbarsysteme des Prototyps.

UML-Sicht 1: Kontext



Systemgrenze, Bediener und externe Zugänge des implementierten Analyseprototyps.

4. Lösungsstrategie

Configuration besitzt Konfigurationsdaten, Analysis Management besitzt Analysezustände. Vier Worker prüfen Fluid, Thermal, Electrical und Engine Management. Der fachliche Schnitt ist eine Entwurfsannahme; kleine Worker beweisen keine sechs vollständig ausgeprägten Bounded Contexts.

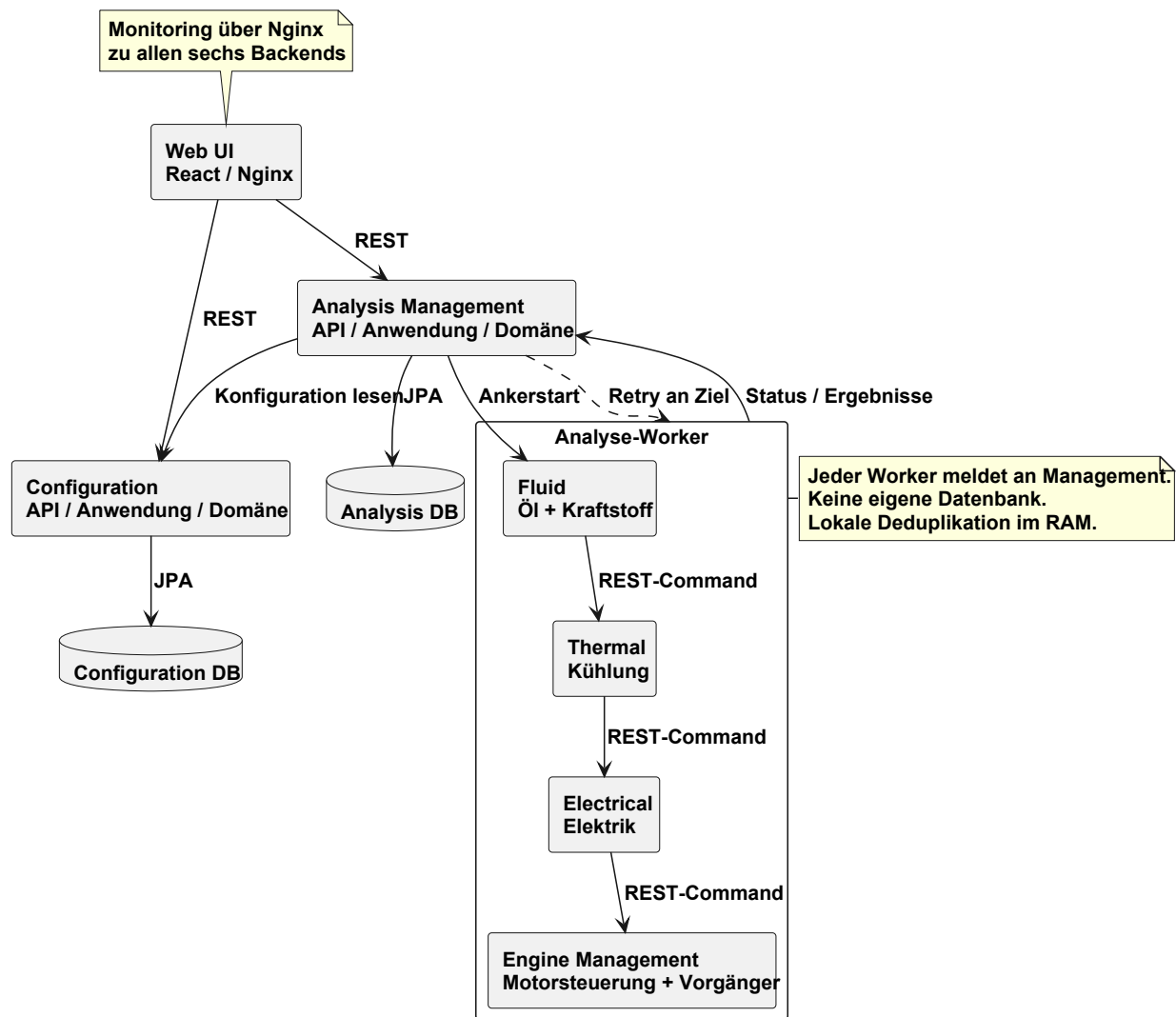
Management startet Fluid; danach geben Worker selbst weiter. Management sammelt Rückmeldungen, berechnet das Gesamtergebnis, überwacht Stillstand und startet angeforderte Retries. Der Normalablauf ist eine sequenzielle REST-Choreographie. Strategische Beziehungen, acht IST-Statements und die konzeptionelle KI-Vision stehen in ddd.md; alle 21 Literaturkriterien in anti-patterns.md.

5. Bausteinsicht

Baustein	Verantwortung
Web UI / Nginx	Bedienung, Weiterleitung sowie Zustands- und Erreichbarkeitsanzeige
Configuration	Equipmentwerte speichern und lesen; eigene Configuration DB
Analysis Management	Analysezustand, Versuchserkennung, Retry und Timeout; eigene Analysis DB
Fluid	Getrennte Resultate für Öl und Kraftstoff
Thermal / Electrical	Kühlung beziehungsweise Elektrik prüfen
Engine Management	Motorsteuerung und vollständige erfolgreiche Vorgänger prüfen

Die zustandsführenden Services enthalten API, Anwendung, Domäne und Infrastruktur. Worker verwenden lokale Command-DTOs und Maps ohne reichhaltige Aggregate. Ein gemeinsames Fachmodell und direkte Zugriffe auf fremde Datenbanken existieren nicht.

UML-Sicht 2: Bausteine



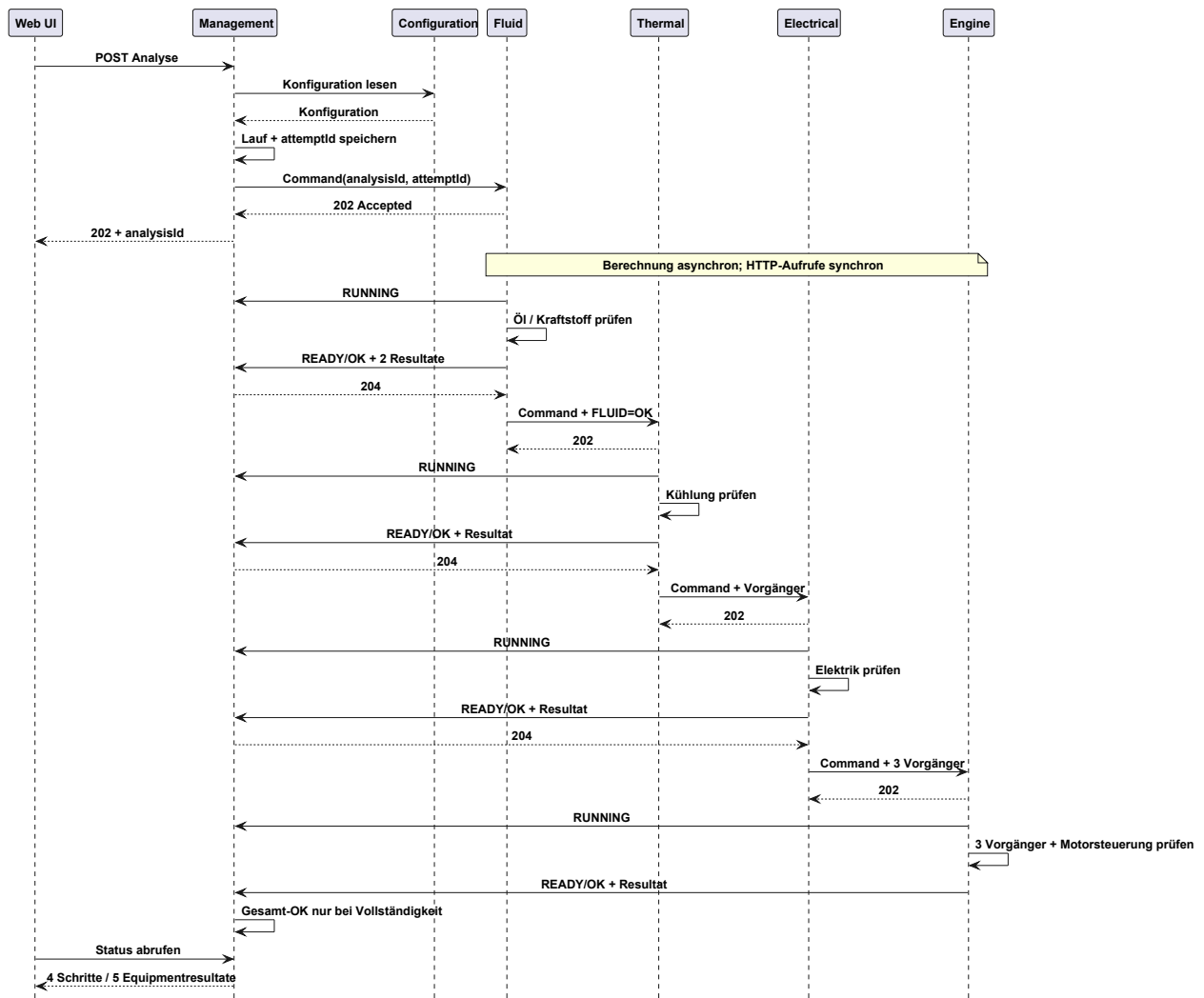
Komponenten, Datenhoheit und Abhängigkeiten zwischen Web UI, Management, Configuration und Analyse-Workern.

6. Laufzeitsicht

- Management lädt die Konfiguration, persistiert einen Lauf mit attemptId und startet Fluid. Der Client erhält 202 Accepted; die Berechnung erfolgt asynchron.
- Jeder Worker meldet RUNNING, danach Equipmentresultate und READY/OK oder FAILED/FAILED. Nach bestätigtem Erfolg startet er den Nachfolger.
- Engine Management verlangt genau ein OK von Fluid, Thermal und Electrical. Gesamt-OK erfordert vier vollständige erfolgreiche Ausführungen.
- Eindeutig abgelehnte Starts werden als technische Fehler gemeldet. Bei unklarer Annahme bleiben Rückmeldungen möglich; der Watchdog beendet den ersten offenen Schritt nach 30 Sekunden ohne Fortschritt.
- Retry setzt Ziel und Nachfolger unter einer Datenbanksperrung auf einen neuen Versuch. Alte oder doppelte terminale Rückmeldungen überschreiben den gültigen Stand nicht.

HTTP-Aufrufe bleiben synchron; HTTP 202 ist keine dauerhafte Arbeitswarteschlange.

UML-Sicht 3: Laufzeit



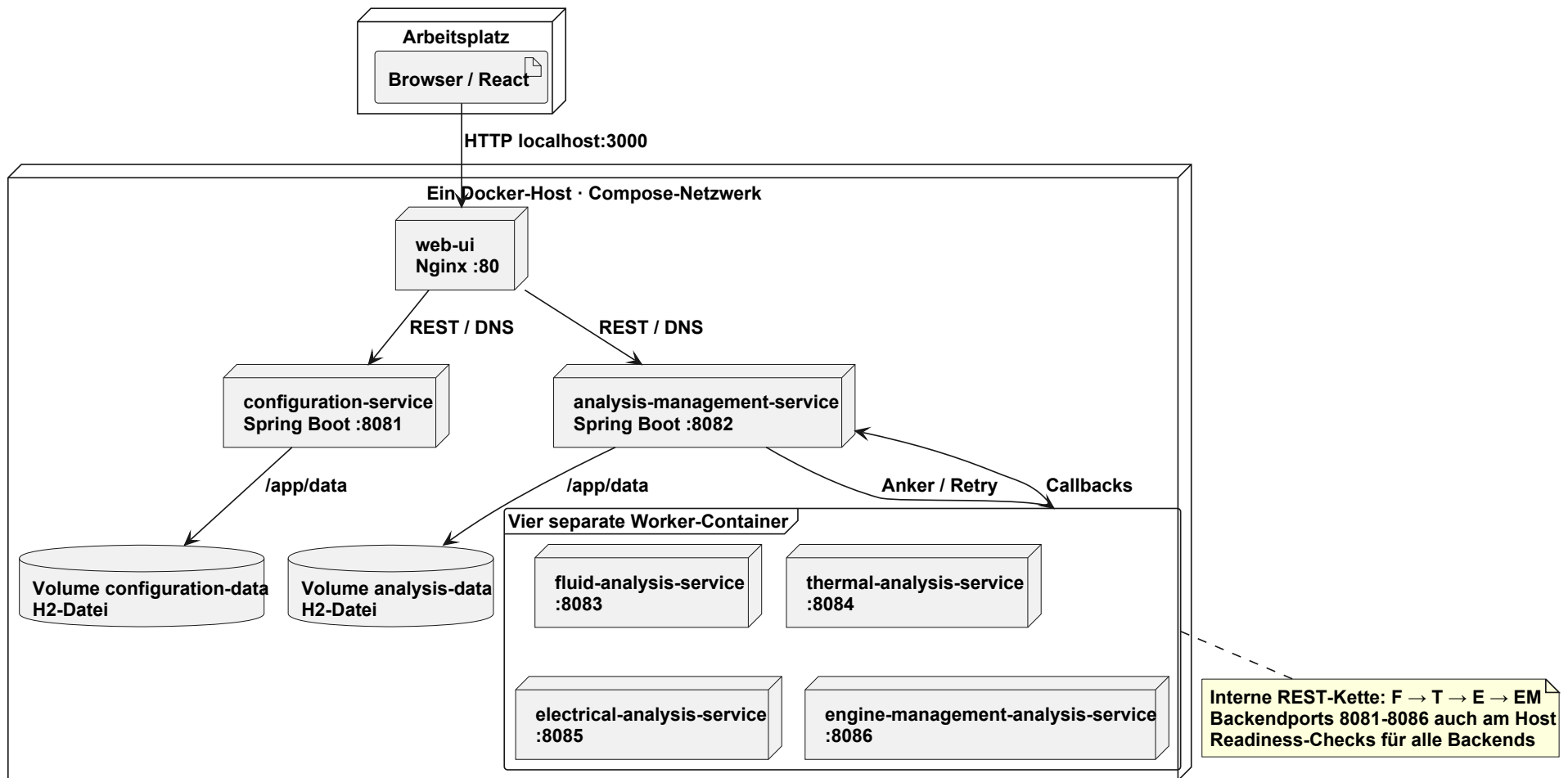
Normalablauf vom Start der Analyse über die Worker-Kette bis zu Status und Gesamtergebnis.

7. Verteilungssicht

Sieben Container laufen auf einem Docker-Host: Web UI 3000:80, Configuration 8081, Management 8082 und Worker 8083-8086. Zwei getrennte Volumes speichern die H2-Dateien. URLs sind konfigurierbar; intern gelten Compose-Servicenamen.

docker-compose.yml baut den lokalen Quellcode. compose.images.yml nutzt vorhandene Registry-Images. Readiness-Checks ordnen den Start; ein offener Breaker macht einen bedienbaren Prozess nicht automatisch unbereit. Austauschbare Container beweisen noch keine unabhängigen Releases oder horizontale Skalierung.

UML-Sicht 4: Verteilung



Lokale Verteilung auf einen Docker-Host mit sieben Containern, Ports, Netzwerk und getrennten Datenvolumes.

8. Querschnittliche Konzepte

Status: PENDING -> RUNNING -> READY/FAILED; technische Startfehler dürfen offene Schritte direkt beenden. Gesamt-OK setzt alle Equipmentresultate voraus. Fachliche Fehler enthalten Einzelresultate; technische Fehler besitzen eine Meldung ohne fingierte Equipmentresultate.

Management hat vier getrennte Ziel-Breaker. Fluid, Thermal und Electrical besitzen je einen nextService-Breaker. HTTP-Verbindungs- und Lesegrenzen betragen 2 beziehungsweise 5 Sekunden. Rückmeldungen werden bei ausgewählten temporären Fehlern höchstens dreimal versucht. Versuchserkennung, Datenbanksperrern und lokale Worker-Deduplikation ergänzen sich; letztere überlebt keinen Prozessneustart.

9. Entwurfsentscheidungen

Entscheidung	Alternative und Konsequenz
Fachlicher Schnitt, getrennte Daten	Ein modularer Monolith wäre betrieblich einfacher. Vier Analyse-Services zeigen den Aufgabenfokus
REST-Choreographie	Ein zentraler Orchestrator vereinfacht die Ablaufübersicht, ein Broker ermöglicht dauerhafte Zustellu
Resilience4j plus Fristen/Versuche	Ein eigener Breaker erhöht den Infrastrukturaufwand. Die Bibliothek ersetzt keine fachlichen Zustand
Compose und H2	Clusterplattform und Serverdatenbank ermöglichen andere Betriebsmodelle, erhöhen aber den Aufw

10. Qualitätsanforderungen und Nachweise

Abnahmefälle: Erfolg, ungültiges Equipment, Ausfall und Retry, fehlender Fortschritt, getrennte Breaker sowie konkurrierende und alte Rückmeldungen. Domain- und Service-Tests prüfen Regeln, HTTP-Clienttests die Verträge, E2E den verbundenen Ablauf. Ausgeführte Prüfungen und Grenzen stehen in testing.md; vorhandene Tests belegen nicht pauschal sämtliche Qualitätsziele.

11. Risiken und Retrospektive

Callbacks und angenommene Arbeit sind nicht dauerhaft gepuffert. Nach einem Management-Ausfall kann ein Worker aufgeben; der Watchdog ermöglicht später Fehleranzeige und manuellen Retry.

Genau-einmal-Ausführung wird nicht garantiert. Der Lauf speichert keinen vollständigen Konfigurationssnapshot; gespeicherte Konfigurationen sind aktuell nur anlegbar und lesbar.

Historische Läufe ohne attemptId bleiben lesbar, erhalten keine erfundenen Equipmentdetails und benötigen für Wiederholung einen neuen Lauf. H2-Dateien, feste Hostports und lokale Deduplikation begrenzen den Mehrinstanzbetrieb. Authentifizierung, API-Versionierung, CI/CD, zentrales Logging und Tracing fehlen.

Lernerfahrung: Container und Breaker allein sichern weder fachliche Konsistenz noch Wiederanlauf. Dafür braucht es Ergebnisregeln, Versuchserkennung und beobachtbare Fehlerpfade.

12. Glossar und Quellen

AnalysisRun: Analyseauftrag; **attemptId:** Versuch des aktiven Kettenabschnitts; **Choreographie:** Weitergabe durch Worker; **Bounded Context:** Gültigkeitsgrenze eines fachlichen Modells.

Grundlagen: Übung 4, Übung 5 und Übung 8, jeweils Seiten 1-3; Fallstudie WirSchiffenDas v3.0. Schirgi/Brenner (2021), *Quality Assurance for Microservice Architectures*, Abschnitte IV.A und IV.B als Bewertungsrahmen. Literaturlinks und Einordnung stehen in anti-patterns.md. Das lokal fehlende Blatt 6 wird nicht als vollständig geprüft ausgegeben.

Dokumentquellen

docs/arc42.md, docs/architecture-views.md, docs/ddd.md, docs/anti-patterns.md, docs/testing.md sowie die vier PlantUML-Quellen in docs/diagrams/.